DYNAMIC QR CODE GENERATOR

Complete Standalone Web Application & Source Code Specification

Exported on: 2026-09-12 | Architecture: HTML5 + React + TailwindCSS + QRCode

Project Overview & Architecture

- Application Name: Dynamic QR Code Generator with Admin Panel
- Formats Supported: SVG (vector scalable) and PNG (raster image) downloads
- Customization: Foreground & background color picker, embedded brand logo upload
- Analytics & Tracking: Live transaction audit logging, IP / country tracking, license validation
- Admin Panel: Secure access with passcode, transaction log management, setting configuration
- Architecture: Zero-dependency runtime, self-contained standalone bundle
- Main Source Code Size: 98,328 characters (1860 lines)

Included Source Code Modules

1. index.html — 1860 lines
Main standalone single-file React application (UI, QR engine, admin panel)

2. package.json — 22 lines
Project configuration, scripts and build definitions

3. metadata.json — 8 lines
Application platform metadata and capability declarations

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Dynamic QR Code Generator with Admin Panel</title>

  <!-- TailwindCSS -->
  <script src="https://cdn.tailwindcss.com"></script>

  <!-- React + ReactDOM -->
  <script src="https://unpkg.com/react@18/umd/react.development.js"></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>

  <!-- QR Code Library -->
  <script src="https://cdn.jsdelivr.net/npm/qrcode/build/qrcode.min.js"></script>

  <!-- jsPDF Library for client-side PDF export fallback -->
  <script src="https://cdnjs.cloudflare.com/ajax/libs/jspdf/2.5.1/jspdf.umd.min.js"></scr...

  <style>
    :root {
      --primary-color: #4f46e5; /* Default indigo-600 */
      --secondary-color: #6366f1; /* Default indigo-500 */
    }
    .qr-wrapper {
      position: relative;
      display: inline-block;
      padding: 1rem; /* Padding around the QR code */
      background-color: var(--qr-bg-color, #ffffff); /* Use CSS variable for background */
      border-radius: 0.75rem; /* Rounded corners for the wrapper */
      box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1), 0 2px 4px -1px rgba(0, 0, 0, 0.06); ...
    }
    .qr-logo {
      position: absolute;
      top: 50%;
      left: 50%;
      width: 25%; /* Slightly smaller logo */
      height: 25%;
      transform: translate(-50%, -50%);
      pointer-events: none;
      object-fit: contain; /* Ensure logo fits without stretching */
      background-color: #ffffff; /* White background for logo area */
      border-radius: 0.5rem; /* Rounded corners for logo */
      padding: 0.25rem; /* Small padding around logo */
      box-shadow: 0 1px 3px 0 rgba(0, 0, 0, 0.1), 0 1px 2px 0 rgba(0, 0, 0, 0.06);
    }
    input[type="color"] {
      -webkit-appearance: none;
      -moz-appearance: none;
      appearance: none;
      width: 2.5rem; /* Standardize color input size */
      height: 2.5rem;
      background-color: transparent;
      border: none;
      cursor: pointer;
    }
    input[type="color"]::-webkit-color-swatch-wrapper {
      padding: 0;
    }
    input[type="color"]::-webkit-color-swatch {
      border: none;
      border-radius: 0.25rem;
    }
    input[type="color"]::-moz-color-swatch-wrapper {
      padding: 0;
    }
    input[type="color"]::-moz-color-swatch {
      border: none;
      border-radius: 0.25rem;
    }
```

```
    .btn-primary {
      background-color: var(--primary-color);
      color: white;
    }
    .btn-primary:hover {
      background-color: var(--secondary-color);
    }
    .text-primary {
      color: var(--primary-color);
    }
    .focus-ring-primary:focus {
      outline: none;
      box-shadow: 0 0 0 2px var(--primary-color);
    }

    /* Print and PDF Export Styles */
    @media print {
      body {
        background-color: #ffffff !important;
        padding: 0 !important;
      }
      header, footer, nav, .no-print, button {
        display: none !important;
      }
      .print-only {
        display: block !important;
      }
      #app {
        max-width: 100% !important;
        box-shadow: none !important;
        border: none !important;
        min-height: auto !important;
      }
      .code-print-view {
        font-family: ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas, monospace !im...
        font-size: 7.5pt !important;
        line-height: 1.25 !important;
        color: #111827 !important;
        white-space: pre-wrap !important;
        word-break: break-all !important;
      }
    }
    @media screen {
      .print-only {
        display: none !important;
      }
    }
  </style>
<link rel="stylesheet" href="/index.css">
</head>

<body class="bg-blue-50 min-h-screen flex flex-col items-center justify-between p-4 sm:p-...

  <div id="app" class="w-full max-w-4xl bg-white rounded-2xl shadow-lg overflow-hidden fl...

  <script type="text/javascript">
    const { useState, useEffect, useCallback, useRef } = React;

    // Utility for debouncing function calls
    const debounce = (func, delay) => {
      let timeout;
      return function(...args) {
        const context = this;
        clearTimeout(timeout);
        timeout = setTimeout(() => func.apply(context, args), delay);
      };
    };

    /**
     * @typedef {object} Transaction
     * @property {string} id
```

```
 * @property {number} timestamp
 * @property {'qrGeneration' | 'licensePurchase'} type
 * @property {string} [email]
 * @property {string} [phone]
 * @property {string} ipAddress
 * @property {string} [country]
 * @property {string} [qrContent]
 * @property {'png' | 'svg'} [format]
 * @property {string} [customerName]
 * @property {string} [customerEmail]
 * @property {string} [customerPhone]
 * @property {string} [purchasePriceUSD]
 * @property {string} [purchaseBtcAddress]
 * @property {string} [purchaseTxId]
 * @property {string} [item]
 * @property {string} [description]
 * @property {'pending' | 'approved' | 'denied'} [status] // Added status for license...
 */

/**
 * @typedef {object} AppSettings
 * @property {string} brandName
 * @property {string} brandLogoUrl
 * @property {string} primaryColor
 * @property {string} secondaryColor
 */

// Icon Components (inlined)
function ChartBarIcon(props) {
  return React.createElement('svg', Object.assign({
    xmlns: 'http://www.w3.org/2000/svg',
    fill: 'none',
    viewBox: '0 0 24 24',
    strokeWidth: '1.5',
    stroke: 'currentColor',
    className: 'w-6 h-6'
  }, props),
    React.createElement('path', { strokeLinecap: 'round', strokeLinejoin: 'round', d:...
    React.createElement('path', { strokeLinecap: 'round', strokeLinejoin: 'round', d:...
  );
}

function QrCodeIcon(props) {
  return React.createElement('svg', Object.assign({
    xmlns: 'http://www.w3.org/2000/svg',
    fill: 'none',
    viewBox: '0 0 24 24',
    strokeWidth: '1.5',
    stroke: 'currentColor',
    className: 'w-6 h-6'
  }, props),
    React.createElement('path', { strokeLinecap: 'round', strokeLinejoin: 'round', d:...
  );
}

function TrashIcon(props) {
  return React.createElement('svg', Object.assign({
    xmlns: 'http://www.w3.org/2000/svg',
    fill: 'none',
    viewBox: '0 0 24 24',
    strokeWidth: '1.5',
    stroke: 'currentColor',
    className: 'w-6 h-6'
  }, props),
    React.createElement('path', { strokeLinecap: 'round', strokeLinejoin: 'round', d:...
  );
}

function CodeBracketIcon(props) {
  return React.createElement('svg', Object.assign({
    xmlns: 'http://www.w3.org/2000/svg',
```

```
        fill: 'none',
        viewBox: '0 0 24 24',
        strokeWidth: '1.5',
        stroke: 'currentColor',
        className: 'w-6 h-6'
      }, props),
        React.createElement('path', { strokeLinecap: 'round', strokeLinejoin: 'round', d:...
      );
    }
    function ClipboardIcon(props) {
      return React.createElement('svg', Object.assign({
        xmlns: 'http://www.w3.org/2000/svg',
        fill: 'none',
        viewBox: '0 0 24 24',
        strokeWidth: '1.5',
        stroke: 'currentColor',
        className: 'w-6 h-6'
      }, props),
        React.createElement('path', { strokeLinecap: 'round', strokeLinejoin: 'round', d:...
      );
    }

    function DocumentTextIcon(props) {
      return React.createElement('svg', Object.assign({
        xmlns: 'http://www.w3.org/2000/svg',
        fill: 'none',
        viewBox: '0 0 24 24',
        strokeWidth: '1.5',
        stroke: 'currentColor',
        className: 'w-6 h-6'
      }, props),
        React.createElement('path', { strokeLinecap: 'round', strokeLinejoin: 'round', d:...
      );
    }

    function ArrowDownTrayIcon(props) {
      return React.createElement('svg', Object.assign({
        xmlns: 'http://www.w3.org/2000/svg',
        fill: 'none',
        viewBox: '0 0 24 24',
        strokeWidth: '1.5',
        stroke: 'currentColor',
        className: 'w-6 h-6'
      }, props),
        React.createElement('path', { strokeLinecap: 'round', strokeLinejoin: 'round', d:...
      );
    }

    function PrinterIcon(props) {
      return React.createElement('svg', Object.assign({
        xmlns: 'http://www.w3.org/2000/svg',
        fill: 'none',
        viewBox: '0 0 24 24',
        strokeWidth: '1.5',
        stroke: 'currentColor',
        className: 'w-6 h-6'
      }, props),
        React.createElement('path', { strokeLinecap: 'round', strokeLinejoin: 'round', d:...
      );
    }

    function CheckIcon(props) {
      return React.createElement('svg', Object.assign({
        xmlns: 'http://www.w3.org/2000/svg',
        fill: 'none',
        viewBox: '0 0 24 24',
        strokeWidth: '2',
        stroke: 'currentColor',
        className: 'w-6 h-6'
      }, props),
        React.createElement('path', { strokeLinecap: 'round', strokeLinejoin: 'round', d:...
```

```
      );
    }


    /**
     * Reusable Color Input Component
     * @param {object} props
     * @param {string} props.value
     * @param {function(string): void} props.onChange
     * @param {string} props.label
     * @param {boolean} [props.disabled]
     * @param {string} [props.className]
     */
    function ColorInput(props) {
      const { value, onChange, label, disabled, className } = props;
      return React.createElement('div', { className: `flex items-center space-x-2 ${class...
        React.createElement('label', { className: 'font-medium text-gray-700' }, label),
        React.createElement('input', {
          type: 'text',
          value: value,
          onChange: (e) => onChange(e.target.value),
          disabled: disabled,
          className: 'w-24 p-2 border border-gray-300 rounded-md shadow-sm focus:ring-blu...
        }),
        React.createElement('input', {
          type: 'color',
          value: value,
          onChange: (e) => onChange(e.target.value),
          disabled: disabled,
          className: 'h-8 w-8 rounded-md border-none cursor-pointer disabled:opacity-70 d...
        })
      );
    }

    /**
     * @param {object} props
     * @param {(transaction: Omit<Transaction, 'id' | 'timestamp' | 'ipAddress' | 'type' ...
     * @param {AppSettings} props.appSettings
     * @param {() => void} [props.onNavigateToSourceCode]
     */
    function QRCodeGenerator(props) {
      const { onTrackTransaction, appSettings, onNavigateToSourceCode } = props;
      const { brandLogoUrl, primaryColor, secondaryColor } = appSettings;

      const [email, setEmail] = useState('');
      const [phone, setPhone] = useState('');
      const [emailError, setEmailError] = useState('');
      const [phoneError, setPhoneError] = useState('');
      const [hasEnteredUserData, setHasEnteredUserData] = useState(false);

      const [qrContent, setQrContent] = useState('https://google.com');
      const [qrDataUrl, setQrDataUrl] = useState('');
      const [logoUrl, setLogoUrl] = useState(''); // User uploaded logo
      const [foregroundColor, setForegroundColor] = useState('#000000');
      const [backgroundColor, setBackgroundColor] = useState('#ffffff');
      const [contentError, setContentError] = useState('');

      const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
      const phoneRegex = /^[0-9\s\-\(\)\+]+$/; // Allows numbers, spaces, hyphens, parent...

      const validateEmail = (inputEmail) => {
        if (!inputEmail) return 'Email is required.';
        if (!emailRegex.test(inputEmail)) return 'Invalid email format.';
        return '';
      };

      const validatePhone = (inputPhone) => {
        if (!inputPhone) return 'Phone number is required.';
        if (!phoneRegex.test(inputPhone)) return 'Invalid phone number format.';
        return '';
      };
```

```
356
357       const handleEmailChange = (e) => {
358         const value = e.target.value;
359         setEmail(value);
360         setEmailError(validateEmail(value));
361       };
362
363       const handlePhoneChange = (e) => {
364         const value = e.target.value;
365         setPhone(value);
366         setPhoneError(validatePhone(value));
367       };
368
369       const handleProceedToGenerator = () => {
370         const emailErr = validateEmail(email);
371         const phoneErr = validatePhone(phone);
372         setEmailError(emailErr);
373         setPhoneError(phoneErr);
374
375         if (!emailErr && !phoneErr) {
376           setHasEnteredUserData(true);
377         }
378       };
379
380       const handleLogoUpload = (e) => {
381         const file = e.target.files[0];
382         if (!file) return;
383
384         const reader = new FileReader();
385         reader.onload = () => setLogoUrl(reader.result);
386         reader.readAsDataURL(file);
387       };
388
389       const debouncedSetQrContent = useCallback(
390         debounce((value) => {
391           setQrContent(value);
392           if (!value) setContentError('QR content cannot be empty.');
393           else setContentError('');
394         }, 300),
395         []
396       );
397
398       // Generate QR Automatically
399       useEffect(() => {
400         if (!qrContent) {
401           setQrDataUrl('');
402           return;
403         }
404         QRCode.toDataURL(qrContent, {
405           margin: 2,
406           width: 500,
407           color: {
408             dark: foregroundColor,
409             light: backgroundColor
410           }
411         })
412           .then(setQrDataUrl)
413           .catch(err => {
414             console.error("Error generating QR code:", err);
415             setQrDataUrl('');
416           });
417
418           // Set custom CSS property for QR wrapper background
419           document.documentElement.style.setProperty('--qr-bg-color', backgroundColor);
420
421       }, [qrContent, foregroundColor, backgroundColor]);
422
423
424       const downloadQRCode = useCallback(async (format) => {
425         if (!qrDataUrl || !hasEnteredUserData || contentError) {
426           alert('Please ensure valid user data and QR content are provided before downloa...
```

```
          return;
        }

        if (format === 'png') {
          const link = document.createElement('a');
          link.download = `qr_${Date.now()}.png`;
          link.href = qrDataUrl;
          link.click();
        } else if (format === 'svg') {
          try {
            const svg = await QRCode.toString(qrContent, {
              type: 'svg',
              color: {
                dark: foregroundColor,
                light: backgroundColor
              }
            });
            const blob = new Blob([svg], { type: 'image/svg+xml' });
            const url = URL.createObjectURL(blob);

            const a = document.createElement('a');
            a.href = url;
            a.download = `qr_${Date.now()}.svg`;
            a.click();

            URL.revokeObjectURL(url);
          } catch (error) {
            console.error("Error generating SVG:", error);
            alert("Failed to generate SVG. Please try again.");
          }
        }

        onTrackTransaction({ email, phone, qrContent, format });
      }, [qrDataUrl, qrContent, email, phone, hasEnteredUserData, onTrackTransaction, for...

      return React.createElement('div', { className: 'p-6 md:p-10 flex flex-col items-cen...
        // User Data Input Section
        React.createElement('div', { className: 'w-full max-w-lg bg-white p-6 rounded-xl ...
          React.createElement('h2', { className: 'text-2xl font-bold text-gray-800 mb-6 t...

          React.createElement('div', { className: 'mb-4' },
            React.createElement('label', { htmlFor: 'emailInput', className: 'block text-...
            React.createElement('input', {
              id: 'emailInput',
              type: 'email',
              value: email,
              onChange: handleEmailChange,
              disabled: hasEnteredUserData, // Disable if already entered
              className: `w-full p-3 border ${emailError ? 'border-red-500' : 'border-gra...
              placeholder: 'your.email@example.com',
              'aria-invalid': !!emailError,
              'aria-describedby': emailError ? 'email-error' : null
            }),
            emailError && React.createElement('p', { id: 'email-error', className: 'mt-1 ...
          ),

          React.createElement('div', { className: 'mb-6' },
            React.createElement('label', { htmlFor: 'phoneInput', className: 'block text-...
            React.createElement('input', {
              id: 'phoneInput',
              type: 'tel', // Changed to tel for semantic correctness
              value: phone,
              onChange: handlePhoneChange,
              disabled: hasEnteredUserData, // Disable if already entered
              className: `w-full p-3 border ${phoneError ? 'border-red-500' : 'border-gra...
              placeholder: '+1 (555) 123-4567',
              'aria-invalid': !!phoneError,
              'aria-describedby': phoneError ? 'phone-error' : null
            }),
            phoneError && React.createElement('p', { id: 'phone-error', className: 'mt-1 ...
          ),
```

```
498
499           !hasEnteredUserData && React.createElement('button', { // Only show "Proceed" i...
500             onClick: handleProceedToGenerator,
501             disabled: !!emailError || !!phoneError || !email || !phone, // Disable if err...
502             className: `w-full font-semibold py-3 px-4 rounded-lg shadow-md focus:outline...
503             style: { backgroundColor: primaryColor, '--tw-ring-color': primaryColor } // ...
504           }, 'Proceed to Generator'),
505
506           hasEnteredUserData && React.createElement('button', { // Add an "Edit Details" ...
507             onClick: () => setHasEnteredUserData(false), // Re-enable inputs
508             className: 'w-full bg-gray-200 text-gray-700 font-semibold py-3 px-4 rounded-...
509           }, 'Edit Details')
510         ),
511
512         // Main QR Generator Section
513         // Changed to flex-col only to ensure vertical stacking
514         React.createElement('div', { className: 'w-full max-w-xl flex flex-col items-cent...
515           React.createElement('div', { className: 'w-full p-6 bg-gray-50 rounded-xl shado...
516             React.createElement('h2', { className: 'text-2xl font-bold text-gray-800 mb-6...
517
518             React.createElement('div', { className: 'mb-4' },
519               React.createElement('label', { htmlFor: 'qrContentInput', className: 'block...
520               React.createElement('textarea', {
521                 id: 'qrContentInput',
522                 value: qrContent,
523                 onChange: (e) => debouncedSetQrContent(e.target.value),
524                 rows: 3,
525                 disabled: !hasEnteredUserData, // Disabled until user data is entered
526                 className: `w-full p-3 border ${contentError ? 'border-red-500' : 'border...
527                 placeholder: 'e.g., https://yourwebsite.com or Your custom text',
528                 'aria-invalid': !!contentError,
529                 'aria-describedby': contentError ? 'content-error' : null
530               }),
531               contentError && React.createElement('p', { id: 'content-error', className: ...
532             ),
533
534             React.createElement('div', { className: 'mb-6' },
535               React.createElement('label', { htmlFor: 'logoUpload', className: 'block tex...
536               React.createElement('input', {
537                 id: 'logoUpload',
538                 type: 'file',
539                 accept: 'image/*',
540                 onChange: handleLogoUpload,
541                 disabled: !hasEnteredUserData, // Disabled
542                 className: 'w-full text-sm text-gray-500 file:mr-4 file:py-2 file:px-4 fi...
543               })
544             ),
545
546             React.createElement('div', { className: 'flex flex-col sm:flex-row justify-be...
547               React.createElement(ColorInput, {
548                 label: 'Foreground Color',
549                 value: foregroundColor,
550                 onChange: setForegroundColor,
551                 disabled: !hasEnteredUserData, // Pass disabled prop to ColorInput
552                 className: 'w-full sm:w-1/2'
553               }),
554               React.createElement(ColorInput, {
555                 label: 'Background Color',
556                 value: backgroundColor,
557                 onChange: setBackgroundColor,
558                 disabled: !hasEnteredUserData, // Pass disabled prop to ColorInput
559                 className: 'w-full sm:w-1/2'
560               })
561             ),
562
563             React.createElement('div', { className: 'flex flex-col sm:flex-row gap-4' },
564               React.createElement('button', {
565                 onClick: () => downloadQRCode('png'),
566                 disabled: !hasEnteredUserData || !qrDataUrl || contentError, // Disabled
567                 className: `flex-1 font-semibold py-3 px-4 rounded-lg shadow-md focus:out...
568                 style: { backgroundColor: primaryColor, '--tw-ring-color': primaryColor }
```

```
              },
                React.createElement('span', null, 'Download PNG')
              ),
              React.createElement('button', {
                onClick: () => downloadQRCode('svg'),
                disabled: !hasEnteredUserData || !qrDataUrl || contentError, // Disabled
                className: `flex-1 bg-green-600 text-white font-semibold py-3 px-4 rounde...
              },
                React.createElement('span', null, 'Download SVG')
              )
            ),

            React.createElement('div', { className: 'mt-6 pt-5 border-t border-gray-200 f...
              React.createElement('div', { className: 'flex items-center gap-2 text-gray-...
                React.createElement(DocumentTextIcon, { className: 'w-4 h-4 text-indigo-6...
                React.createElement('span', null, 'Need the source code for this applicat...
              ),
              React.createElement('button', {
                onClick: () => onNavigateToSourceCode && onNavigateToSourceCode(),
                className: 'text-indigo-600 hover:text-indigo-800 font-semibold hover:und...
              },
                React.createElement('span', null, 'Download Source Code (PDF)'),
                React.createElement('span', null, '?')
              )
            )
          ),

          // QR Code Preview Section
          // Removed flex-none, lg:w-72, order classes for vertical stacking
          React.createElement('div', { className: 'w-full p-6 bg-gray-50 rounded-xl shado...
            React.createElement('h2', { className: 'text-xl font-bold text-gray-800 mb-4'...
            React.createElement('div', { className: 'qr-wrapper', style: { '--qr-bg-color...
              qrDataUrl
                ? React.createElement('img', { src: qrDataUrl, alt: 'Generated QR Code', ...
                : React.createElement('div', { className: 'w-full h-full bg-gray-200 flex...
              (logoUrl || brandLogoUrl)
                ? React.createElement('img', { src: logoUrl || brandLogoUrl, alt: 'Logo',...
                : null
            )
          )
        )
      );
    }

    /**
     * AdminLogin Component
     * @param {object} props
     * @param {(password: string) => void} props.onLogin
     * @param {() => void} props.onResetPassword
     * @param {string} props.error
     * @param {AppSettings} props.appSettings
     */
    function AdminLogin(props) {
      const { onLogin, onResetPassword, error, appSettings } = props;
      const { primaryColor } = appSettings;
      const [password, setPassword] = useState('');
      const [resetRequested, setResetRequested] = useState(false);

      const handleLoginAttempt = () => {
        if (password) {
          onLogin(password);
        } else {
          props.onLogin(''); // Pass empty to trigger App's specific error or default
        }
      };

      const handleResetRequest = () => {
        if (window.confirm('Are you sure you want to reset the Admin password to the defa...
          onResetPassword();
          setResetRequested(true);
          setTimeout(() => setResetRequested(false), 3000); // Clear message
```

```
        }
      };

      return React.createElement('div', { className: 'p-6 md:p-10 flex flex-col items-cen...
        React.createElement('div', { className: 'w-full max-w-md bg-white p-8 rounded-xl ...
          React.createElement('h2', { className: 'text-3xl font-bold text-gray-800 mb-6' ...
          React.createElement('p', { className: 'text-gray-600 mb-6' }, 'Enter the Admin ...

          React.createElement('div', { className: 'mb-4' },
            React.createElement('input', {
              type: 'password',
              value: password,
              onChange: (e) => { setPassword(e.target.value); },
              onKeyPress: (e) => { if (e.key === 'Enter') handleLoginAttempt(); },
              className: `w-full p-3 border ${error ? 'border-red-500' : 'border-gray-300...
              style: { '--tw-ring-color': primaryColor, borderColor: error ? '#ef4444' : ...
              placeholder: 'Admin Password',
              'aria-invalid': !!error,
              'aria-describedby': error ? 'admin-login-error' : null
            }),
            error && React.createElement('p', { id: 'admin-login-error', className: 'mt-2...
          ),

          React.createElement('button', {
            onClick: handleLoginAttempt,
            className: `w-full font-semibold py-3 px-4 rounded-lg shadow-md focus:outline...
            style: { backgroundColor: primaryColor, '--tw-ring-color': primaryColor }
          }, 'Login'),

          React.createElement('div', { className: 'mt-6 pt-4 border-t border-gray-200' },
            React.createElement('button', {
              onClick: handleResetRequest,
              className: 'text-sm text-gray-600 hover:text-red-700 hover:underline transi...
            }, 'Forgot Password? Reset to Default'),
            resetRequested && React.createElement('p', { className: 'mt-2 text-sm text-gr...
          )
        )
      );
    }


    /**
     * @param {object} props
     * @param {Transaction[]} props.transactions
     * @param {function(): void} props.onClearHistory
     * @param {(id: string, newStatus: 'approved' | 'denied') => void} props.onUpdateLice...
     * @param {() => void} props.onLogout
     * @param {() => void} props.onResetPassword
     * @param {AppSettings} props.appSettings
     * @param {(settings: AppSettings) => void} props.onSaveAppSettings
     */
    function AdminPanel(props) {
      const { transactions, onClearHistory, onUpdateLicenseStatus, onLogout, onResetPassw...
      const { brandName, brandLogoUrl, primaryColor, secondaryColor } = appSettings;
      const [copied, setCopied] = useState(false);

      const [settingsBrandName, setSettingsBrandName] = useState(brandName);
      const [settingsBrandLogoUrl, setSettingsBrandLogoUrl] = useState(brandLogoUrl);
      const [settingsPrimaryColor, setSettingsPrimaryColor] = useState(primaryColor);
      const [settingsSecondaryColor, setSettingsSecondaryColor] = useState(secondaryColor);

      useEffect(() => {
        setSettingsBrandName(brandName);
        setSettingsBrandLogoUrl(brandLogoUrl);
        setSettingsPrimaryColor(primaryColor);
        setSettingsSecondaryColor(secondaryColor);
      }, [brandName, brandLogoUrl, primaryColor, secondaryColor]);


      const totalQrGenerations = transactions.filter(t => t.type === 'qrGeneration').length;
      const pngCount = transactions.filter(t => t.type === 'qrGeneration' && t.format ===...
```

```
      const svgCount = transactions.filter(t => t.type === 'qrGeneration' && t.format ===...
      const totalLicensePurchases = transactions.filter(t => t.type === 'licensePurchase'...


      const handleCopySourceCode = () => {
        const sourceCode = document.documentElement.outerHTML;
        navigator.clipboard.writeText(sourceCode).then(() => {
          setCopied(true);
          setTimeout(() => setCopied(false), 2000); // Reset after 2 seconds
        }).catch(err => {
          console.error('Failed to copy source code:', err);
          alert('Failed to copy source code.');
        });
      };

      const handleSaveSettings = () => {
        onSaveAppSettings({
          brandName: settingsBrandName,
          brandLogoUrl: settingsBrandLogoUrl,
          primaryColor: settingsPrimaryColor,
          secondaryColor: settingsSecondaryColor,
        });
        alert('App settings saved!');
      };

      const getStatusBadge = (status) => {
        switch (status) {
          case 'approved':
            return React.createElement('span', { className: 'px-2 inline-flex text-xs lea...
          case 'denied':
            return React.createElement('span', { className: 'px-2 inline-flex text-xs lea...
          case 'pending':
          default:
            return React.createElement('span', { className: 'px-2 inline-flex text-xs lea...
        }
      };


      return React.createElement('div', { className: 'p-6 md:p-10 flex flex-col gap-8' },
        React.createElement('h2', { className: 'text-3xl font-extrabold text-gray-900 mb-...

        // Admin Controls
        React.createElement('div', { className: 'flex justify-center gap-4 mb-8' },
          React.createElement('button', {
            onClick: onLogout,
            className: 'bg-gray-700 text-white font-medium py-2 px-4 rounded-lg shadow-md...
          }, 'Logout'),
          React.createElement('button', {
            onClick: onResetPassword,
            className: 'bg-yellow-500 text-white font-medium py-2 px-4 rounded-lg shadow-...
          }, 'Reset Admin Password')
        ),

        // App Settings Section
        React.createElement('div', { className: 'bg-white p-6 rounded-xl shadow-lg border...
          React.createElement('h3', { className: 'text-xl font-semibold text-gray-800 mb-...
          React.createElement('div', { className: 'grid grid-cols-1 md:grid-cols-2 gap-4' },
            React.createElement('div', null,
              React.createElement('label', { className: 'block text-sm font-medium text-g...
              React.createElement('input', {
                type: 'text',
                value: settingsBrandName,
                onChange: (e) => setSettingsBrandName(e.target.value),
                className: 'w-full p-2 border border-gray-300 rounded-lg shadow-sm focus-...
                style: { '--tw-ring-color': primaryColor }
              })
            ),
            React.createElement('div', null,
              React.createElement('label', { className: 'block text-sm font-medium text-g...
              React.createElement('input', {
                type: 'text',
```

```
                value: settingsBrandLogoUrl,
                onChange: (e) => setSettingsBrandLogoUrl(e.target.value),
                className: 'w-full p-2 border border-gray-300 rounded-lg shadow-sm focus-...
                style: { '--tw-ring-color': primaryColor }
              })
            ),
            React.createElement(ColorInput, {
              label: 'Primary Color',
              value: settingsPrimaryColor,
              onChange: setSettingsPrimaryColor,
              className: 'w-full'
            }),
            React.createElement(ColorInput, {
              label: 'Secondary Color',
              value: settingsSecondaryColor,
              onChange: setSettingsSecondaryColor,
              className: 'w-full'
            })
          ),
          React.createElement('button', {
            onClick: handleSaveSettings,
            className: `mt-6 w-full font-semibold py-3 px-4 rounded-lg shadow-md focus:ou...
            style: { backgroundColor: primaryColor, '--tw-ring-color': primaryColor }
          }, 'Save Settings')
        ),


        // Stats Section
        React.createElement('div', { className: 'grid grid-cols-1 md:grid-cols-4 gap-4 mb...
          React.createElement('div', { className: 'bg-white p-5 rounded-lg shadow border ...
            React.createElement('div', { className: `text-4xl font-bold text-primary`, st...
            React.createElement('div', { className: 'text-gray-600 text-sm font-medium mt...
          ),
          React.createElement('div', { className: 'bg-white p-5 rounded-lg shadow border ...
            React.createElement('div', { className: `text-4xl font-bold text-primary`, st...
            React.createElement('div', { className: 'text-gray-600 text-sm font-medium mt...
          ),
          React.createElement('div', { className: 'bg-white p-5 rounded-lg shadow border ...
            React.createElement('div', { className: `text-4xl font-bold text-primary`, st...
            React.createElement('div', { className: 'text-gray-600 text-sm font-medium mt...
          ),
          React.createElement('div', { className: 'bg-white p-5 rounded-lg shadow border ...
            React.createElement('div', { className: `text-4xl font-bold text-green-600` }...
            React.createElement('div', { className: 'text-gray-600 text-sm font-medium mt...
          )
        ),

        // History Table
        React.createElement('div', { className: 'bg-white p-6 rounded-xl shadow-lg border...
          React.createElement('div', { className: 'flex justify-between items-center mb-6...
            React.createElement('h3', { className: 'text-xl font-semibold text-gray-800' ...
            React.createElement('button', {
              onClick: onClearHistory,
              className: 'bg-red-500 text-white font-medium py-2 px-4 rounded-lg shadow-m...
            },
              React.createElement(TrashIcon, { className: 'w-5 h-5' }),
              'Clear History'
            )
          ),

          transactions.length === 0
            ? React.createElement('p', { className: 'text-gray-500 text-center py-8' }, '...
            : React.createElement('div', { className: 'overflow-x-auto' },
                React.createElement('table', { className: 'min-w-full divide-y divide-gra...
                  React.createElement('thead', { className: 'bg-gray-50' },
                    React.createElement('tr', null,
                      React.createElement('th', { scope: 'col', className: 'px-6 py-3 tex...
                      React.createElement('th', { scope: 'col', className: 'px-6 py-3 tex...
                      React.createElement('th', { scope: 'col', className: 'px-6 py-3 tex...
                      React.createElement('th', { scope: 'col', className: 'px-6 py-3 tex...
                      React.createElement('th', { scope: 'col', className: 'px-6 py-3 tex...
```

```
                    React.createElement('th', { scope: 'col', className: 'px-6 py-3 tex...
                    React.createElement('th', { scope: 'col', className: 'px-6 py-3 tex...
                    React.createElement('th', { scope: 'col', className: 'px-6 py-3 tex...
                    React.createElement('th', { scope: 'col', className: 'px-6 py-3 tex...
                  )
                ),
                React.createElement('tbody', { className: 'bg-white divide-y divide-gra...
                  transactions.map((t) =>
                    React.createElement('tr', { key: t.id },
                      React.createElement('td', { className: 'px-6 py-4 whitespace-nowr...
                      React.createElement('td', { className: 'px-6 py-4 whitespace-nowr...
                        React.createElement('span', { className: `px-2 inline-flex text...
                          t.type === 'qrGeneration' ? 'QR Gen' : 'License'
                        )
                      ),
                      React.createElement('td', { className: 'px-6 py-4 whitespace-nowr...
                      React.createElement('td', { className: 'px-6 py-4 whitespace-nowr...
                      React.createElement('td', { className: 'px-6 py-4 whitespace-nowr...
                      React.createElement('td', { className: 'px-6 py-4 text-sm text-gr...
                        t.type === 'qrGeneration'
                          ? (t.qrContent && t.qrContent.length > 50 ? t.qrContent.subst...
                          : t.item || 'N/A'
                      ),
                      React.createElement('td', { className: 'px-6 py-4 whitespace-nowr...
                        t.type === 'qrGeneration'
                          ? React.createElement('span', { className: `px-2 inline-flex ...
                          : React.createElement('span', { className: 'px-2 inline-flex ...
                      ),
                      // Status Column
                      React.createElement('td', { className: 'px-6 py-4 whitespace-nowr...
                        t.type === 'licensePurchase' ? getStatusBadge(t.status) : 'N/A'
                      ),
                      // Actions Column
                      React.createElement('td', { className: 'px-6 py-4 whitespace-nowr...
                        t.type === 'licensePurchase' && t.status === 'pending' && React...
                          React.createElement('button', {
                            onClick: () => onUpdateLicenseStatus(t.id, 'approved'),
                            className: 'bg-green-500 hover:bg-green-600 text-white py-1...
                          }, 'Approve'),
                          React.createElement('button', {
                            onClick: () => onUpdateLicenseStatus(t.id, 'denied'),
                            className: 'bg-red-500 hover:bg-red-600 text-white py-1 px-...
                          }, 'Deny')
                        ),
                        t.type === 'licensePurchase' && t.status !== 'pending' && React...
                      )
                    )
                  )
                )
              )
            )
          )
        ),

        // Share App Source Code Section
        React.createElement('div', { className: 'bg-white p-6 rounded-xl shadow-lg border...
          React.createElement('h3', { className: 'text-xl font-semibold text-gray-800 mb-...
          React.createElement('p', { className: 'text-gray-600 mb-4' }, 'You can copy the...
          React.createElement('textarea', {
            readOnly: true,
            value: document.documentElement.outerHTML, // Get full HTML of the page
            className: 'w-full h-96 p-4 border border-gray-300 rounded-lg bg-gray-50 font...
            style: { whiteSpace: 'pre' } // Preserve whitespace for code formatting
          }),
            React.createElement('button', {
            onClick: handleCopySourceCode,
            className: 'mt-4 w-full bg-indigo-600 text-white font-semibold py-3 px-4 roun...
          },
            React.createElement(CodeBracketIcon, { className: 'w-5 h-5' }),
            copied ? 'Copied!' : 'Copy to Clipboard'
          )
        )
```

```
      );
    }

    /**
     * Component for the "For Sale" page
     * @param {object} props
     * @param {(licenseTransaction: Omit<Transaction, 'id' | 'type' | 'status'> & { ipAdd...
     * @param {AppSettings} props.appSettings
     */
    function ForSalePage(props) {
      const { onTrackLicensePurchase, appSettings } = props;
      const { brandName, primaryColor } = appSettings;

      const [showDepositPage, setShowDepositPage] = useState(false);
      const [isUserInfoSubmitted, setIsUserInfoSubmitted] = useState(false);
      const [customerName, setCustomerName] = useState('');
      const [customerEmail, setCustomerEmail] = useState('');
      const [customerPhone, setCustomerPhone] = useState('');
      const [nameError, setNameError] = useState('');
      const [emailError, setEmailError] = useState('');
      const [phoneError, setPhoneError] = useState('');

      const [ipAddress, setIpAddress] = useState('Fetching...');
      const [isLoadingIp, setIsLoadingIp] = useState(true);
      const [ipError, setIpError] = useState('');

      const [country, setCountry] = useState('Fetching...'); // New state for country
      const [countryError, setCountryError] = useState(''); // New state for country error

      const [depositTimestamp, setDepositTimestamp] = useState(null);
      const [transactionId, setTransactionId] = useState(null);
      const [btcQrDataUrl, setBtcQrDataUrl] = useState('');
      const [btcQrError, setBtcQrError] = useState('');

      const [copiedWalletAddress, setCopiedWalletAddress] = useState(false);
      const [copiedAllDetails, setCopiedAllDetails] = useState(false);

      const btcWalletAddress = '174oU95LyW2bXF6XRyLshqGvqR6fQTMca8';
      const priceUSD = '2000';
      const itemDescription = brandName; // Use brandName for item
      const productDescription = 'Full source code with Admin Panel, QR customization, an...


      const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
      const phoneRegex = /^[0-9\s\-\(\)\+]+$/;

      const validateName = (name) => (!name ? 'Name is required.' : '');
      const validateEmail = (email) => (!email ? 'Email is required.' : !emailRegex.test(...
      const validatePhone = (phone) => (!phone ? 'Phone is required.' : !phoneRegex.test(...

      const handleNameChange = (e) => {
        const value = e.target.value;
        setCustomerName(value);
        setNameError(validateName(value));
      };
      const handleEmailChange = (e) => {
        const value = e.target.value;
        setCustomerEmail(value);
        setEmailError(validateEmail(value));
      };
      const handlePhoneChange = (e) => {
        const value = e.target.value;
        setCustomerPhone(value);
        setPhoneError(validatePhone(value));
      };

      const handleConfirmUserInfo = () => {
        const nameErr = validateName(customerName);
        const emailErr = validateEmail(customerEmail);
        const phoneErr = validatePhone(customerPhone);

```

```
        setNameError(nameErr);
        setEmailError(emailErr);
        setPhoneError(phoneErr);

        if (!nameErr && !emailErr && !phoneErr) {
          setIsUserInfoSubmitted(true);
          const currentTimestamp = Date.now();
          const currentTransactionId = crypto.randomUUID();
          setDepositTimestamp(currentTimestamp);
          setTransactionId(currentTransactionId);
        }
      };

      useEffect(() => {
        if (isUserInfoSubmitted) {
          // Fetch IP Address and Country
          setIsLoadingIp(true);
          setIpError('');
          setCountryError('');
          setIpAddress('Fetching...');
          setCountry('Fetching...');

          fetch('https://ipapi.co/json/')
            .then(response => {
              if (!response.ok) {
                throw new Error(`HTTP error! status: ${response.status}`);
              }
              return response.json();
            })
            .then(data => {
              const fetchedIp = data.ip || 'N/A';
              const fetchedCountry = data.country_name || 'N/A';
              setIpAddress(fetchedIp);
              setCountry(fetchedCountry);

              // Now that IP and Country are fetched, track the license purchase
              onTrackLicensePurchase({
                customerName,
                customerEmail,
                customerPhone,
                purchasePriceUSD: priceUSD,
                purchaseBtcAddress: btcWalletAddress,
                item: itemDescription,
                description: productDescription,
                timestamp: depositTimestamp,
                transactionId: transactionId,
                ipAddress: fetchedIp,
                country: fetchedCountry,
                status: 'pending', // Default status for new license purchases
              });
            })
            .catch(error => {
              console.error("Failed to fetch IP address or country:", error);
              setIpAddress('Could not fetch IP (client-side)');
              setCountry('Could not fetch Country');
              setIpError('Failed to fetch IP/Country.');
              setCountryError('Failed to fetch Country.');

              // Still track the transaction even if IP/Country fetch fails
              onTrackLicensePurchase({
                customerName,
                customerEmail,
                customerPhone,
                purchasePriceUSD: priceUSD,
                purchaseBtcAddress: btcWalletAddress,
                item: itemDescription,
                description: productDescription,
                timestamp: depositTimestamp,
                transactionId: transactionId,
                ipAddress: 'Could not fetch IP (client-side)',
                country: 'Could not fetch Country',
```

```
                status: 'pending', // Default status for new license purchases
              });
            })
            .finally(() => {
              setIsLoadingIp(false);
            });

          // Generate BTC QR Code
          setBtcQrError('');
          QRCode.toDataURL(`bitcoin:${btcWalletAddress}`, { margin: 2, width: 200 })
            .then(setBtcQrDataUrl)
            .catch(err => {
              console.error("Error generating BTC QR code:", err);
              setBtcQrDataUrl('');
              setBtcQrError('Failed to generate BTC QR code.');
            });
        }
      }, [isUserInfoSubmitted, btcWalletAddress, customerName, customerEmail, customerPho...


      const handleCopyWalletAddress = () => {
        navigator.clipboard.writeText(btcWalletAddress).then(() => {
          setCopiedWalletAddress(true);
          setTimeout(() => setCopiedWalletAddress(false), 2000);
        }).catch(err => {
          console.error('Failed to copy wallet address:', err);
          alert('Failed to copy wallet address.');
        });
      };

      const handleCopyAllDetails = () => {
        const details = `
--- Purchase Details ---
Item: ${itemDescription}
Price: $${priceUSD} USD (equivalent in BTC)
Description: ${productDescription}
BTC Wallet Address: ${btcWalletAddress}
--- Customer Information ---
Name: ${customerName}
Email: ${customerEmail}
Phone: ${customerPhone}
IP Address: ${ipAddress}
Country: ${country}
Timestamp: ${depositTimestamp ? new Date(depositTimestamp).toLocaleString() : 'N/A'}
Transaction ID: ${transactionId || 'N/A'}
--------------------------
Please send the Bitcoin equivalent to the address above.
`;
        navigator.clipboard.writeText(details.trim()).then(() => {
          setCopiedAllDetails(true);
          setTimeout(() => setCopiedAllDetails(false), 2000);
        }).catch(err => {
          console.error('Failed to copy details:', err);
          alert('Failed to copy all details.');
        });
      };


      return React.createElement('div', { className: 'p-6 md:p-10 flex flex-col items-cen...
        !showDepositPage
          ? React.createElement(React.Fragment, null, // Initial Sales Pitch
              React.createElement('h2', { className: 'text-4xl font-extrabold text-gray-9...
              React.createElement('p', { className: 'text-xl text-gray-700 max-w-2xl' },
                'Looking for a fully customizable, dynamic QR Code Generator for your bus...
                React.createElement('strong', { className: 'text-primary', style: { color...
                ' It features user data collection, admin tracking, and seamless QR code ...
              ),
              React.createElement('div', { className: 'mt-8 flex flex-col sm:flex-row gap...
                React.createElement('button', {
                  onClick: () => setShowDepositPage(true),
                  className: `font-semibold py-3 px-8 rounded-lg shadow-md focus:outline-...
```

```
                    style: { backgroundColor: primaryColor, '--tw-ring-color': primaryColor }
                  }, 'Purchase Now')
                ),
                React.createElement('p', { className: 'text-sm text-gray-500 mt-8' },
                  'Serious inquiries only. Price negotiable based on extensive customizatio...
                )
              )
            : !isUserInfoSubmitted
              ? React.createElement('div', { className: 'w-full max-w-lg bg-white p-8 round...
                  React.createElement('h2', { className: 'text-3xl font-bold text-gray-800 ...

                  React.createElement('div', { className: 'mb-4 text-left' },
                    React.createElement('label', { htmlFor: 'customerName', className: 'blo...
                    React.createElement('input', {
                      id: 'customerName',
                      type: 'text',
                      value: customerName,
                      onChange: handleNameChange,
                      className: `w-full p-3 border ${nameError ? 'border-red-500' : 'borde...
                      style: { '--tw-ring-color': primaryColor },
                      placeholder: 'John Doe',
                      'aria-invalid': !!nameError,
                      'aria-describedby': nameError ? 'name-error' : null
                    }),
                    nameError && React.createElement('p', { id: 'name-error', className: 'm...
                  ),

                  React.createElement('div', { className: 'mb-4 text-left' },
                    React.createElement('label', { htmlFor: 'customerEmail', className: 'bl...
                    React.createElement('input', {
                      id: 'customerEmail',
                      type: 'email',
                      value: customerEmail,
                      onChange: handleEmailChange,
                      className: `w-full p-3 border ${emailError ? 'border-red-500' : 'bord...
                      style: { '--tw-ring-color': primaryColor },
                      placeholder: 'your.email@example.com',
                      'aria-invalid': !!emailError,
                      'aria-describedby': emailError ? 'customer-email-error' : null
                    }),
                    emailError && React.createElement('p', { id: 'customer-email-error', cl...
                  ),

                  React.createElement('div', { className: 'mb-6 text-left' },
                    React.createElement('label', { htmlFor: 'customerPhone', className: 'bl...
                    React.createElement('input', {
                      id: 'customerPhone',
                      type: 'tel',
                      value: customerPhone,
                      onChange: handlePhoneChange,
                      className: `w-full p-3 border ${phoneError ? 'border-red-500' : 'bord...
                      style: { '--tw-ring-color': primaryColor },
                      placeholder: '+1 (555) 123-4567',
                      'aria-invalid': !!phoneError,
                      'aria-describedby': phoneError ? 'customer-phone-error' : null
                    }),
                    phoneError && React.createElement('p', { id: 'customer-phone-error', cl...
                  ),

                  React.createElement('button', {
                    onClick: handleConfirmUserInfo,
                    disabled: !!nameError || !!emailError || !!phoneError || !customerName ...
                    className: `w-full font-semibold py-3 px-8 rounded-lg shadow-md focus:o...
                    style: { backgroundColor: primaryColor, '--tw-ring-color': primaryColor }
                  }, 'Confirm Details & Proceed to Deposit'),

                  React.createElement('button', {
                      onClick: () => setShowDepositPage(false),
                      className: 'mt-4 w-full bg-gray-200 text-gray-800 font-semibold py-3 ...
                  }, 'Back to Sales Page')
                )
```

```
            : React.createElement(React.Fragment, null, // BTC Deposit Details Page
                React.createElement('h2', { className: 'text-4xl font-extrabold text-gray...
                React.createElement('p', { className: 'text-xl text-gray-700 max-w-2xl' },
                  'To acquire this application, please send the equivalent of ',
                  React.createElement('strong', { className: 'text-primary', style: { col...
                  ' to the wallet address provided below. Your purchase details are also ...
                ),
                React.createElement('div', { className: 'mt-8 w-full max-w-md flex flex-c...
                  // Your Bitcoin Deposit Information block
                  React.createElement('div', { className: 'bg-gray-100 p-6 rounded-lg sha...
                    React.createElement('h3', { className: 'text-xl font-bold text-gray-8...

                    btcQrDataUrl
                      ? React.createElement('div', { className: 'mb-4 p-2 bg-white rounde...
                          React.createElement('img', { src: btcQrDataUrl, alt: 'Bitcoin W...
                        )
                      : React.createElement('div', { className: 'mb-4 p-2 bg-white rounde...
                          isLoadingIp ? 'Generating QR...' : (btcQrError || 'QR Unavailab...
                        ),

                    React.createElement('p', { className: 'mb-2 text-gray-900 text-center...
                    React.createElement('p', { className: 'mb-4 break-all text-gray-900 t...

                    React.createElement('button', { // Dedicated Copy Wallet Address Button
                      onClick: handleCopyWalletAddress,
                      className: 'mb-6 bg-gray-200 text-gray-700 font-semibold py-2 px-4 ...
                    },
                      React.createElement(ClipboardIcon, { className: 'w-4 h-4' }),
                      copiedWalletAddress ? 'Copied!' : 'Copy Wallet Address'
                    )
                  ),

                  // Your Purchase Details block
                  React.createElement('div', { className: 'bg-gray-100 p-6 rounded-lg sha...
                    React.createElement('h3', { className: 'text-xl font-bold text-gray-8...
                    React.createElement('p', { className: 'mb-2 text-gray-900' }, React.c...
                    React.createElement('p', { className: 'mb-2 text-gray-900' }, React.c...
                    React.createElement('p', { className: 'mb-2 text-gray-900' }, React.c...
                    React.createElement('hr', { className: 'my-4 border-gray-300' }),
                    React.createElement('p', { className: 'mb-2 text-gray-900' }, React.c...
                    React.createElement('p', { className: 'mb-2 text-gray-900' }, React.c...
                    React.createElement('p', { className: 'mb-2 text-gray-900' }, React.c...
                    React.createElement('p', { className: 'mb-2 text-gray-900' }, React.c...
                    React.createElement('p', { className: 'mb-2 text-gray-900' }, React.c...
                    React.createElement('p', { className: 'mb-2 text-gray-900' }, React.c...
                    React.createElement('p', { className: 'mb-4 text-gray-900' }, React.c...

                    React.createElement('button', {
                      onClick: handleCopyAllDetails,
                      className: `mt-4 w-full font-semibold py-3 px-4 rounded-lg shadow-m...
                      style: { backgroundColor: primaryColor, '--tw-ring-color': primaryC...
                    },
                      React.createElement(ClipboardIcon, { className: 'w-5 h-5' }),
                      copiedAllDetails ? 'Copied All Details!' : 'Copy All Details'
                    )
                  )
                ),
                React.createElement('div', { className: 'mt-8 flex flex-col sm:flex-row g...
                    React.createElement('button', {
                        onClick: () => setIsUserInfoSubmitted(false),
                        className: 'bg-gray-200 text-gray-800 font-semibold py-3 px-8 rou...
                    }, 'Back to Edit Details'),
                    React.createElement('button', {
                        onClick: () => setShowDepositPage(false),
                        className: 'bg-gray-200 text-gray-800 font-semibold py-3 px-8 rou...
                    }, 'Back to Sales Page')
                )
              )
        );
    }

```

```
/**
 * SourceCodeExport Component
 * Displays code tabs, downloadable PDF trigger, printable preview, and raw HTML export
 * @param {object} props
 * @param {AppSettings} props.appSettings
 * @param {() => void} props.onNavigateToGenerator
 */
function SourceCodeExport(props) {
  const { appSettings, onNavigateToGenerator } = props;
  const { primaryColor } = appSettings;

  const [selectedFile, setSelectedFile] = useState('index.html');
  const [files, setFiles] = useState({
    'index.html': '<!-- Loading application source code... -->',
    'package.json': '{\n  "status": "loading..."\n}',
    'metadata.json': '{\n  "status": "loading..."\n}',
    'scripts/generate-source-pdf.js': '// Loading PDF generation script...'
  });
  const [searchQuery, setSearchQuery] = useState('');
  const [copied, setCopied] = useState(false);
  const [isDownloadingPdf, setIsDownloadingPdf] = useState(false);
  const [downloadSuccess, setDownloadSuccess] = useState(false);

  useEffect(() => {
    fetch('/index.html')
      .then(res => res.text())
      .then(text => setFiles(prev => ({ ...prev, 'index.html': text })))
      .catch(() => {
        setFiles(prev => ({ ...prev, 'index.html': document.documentElement.outerHTML...
      });

    fetch('/package.json')
      .then(res => res.text())
      .then(text => setFiles(prev => ({ ...prev, 'package.json': text })))
      .catch(() => {});

    fetch('/metadata.json')
      .then(res => res.text())
      .then(text => setFiles(prev => ({ ...prev, 'metadata.json': text })))
      .catch(() => {});

    fetch('/scripts/generate-source-pdf.js')
      .then(res => res.text())
      .then(text => setFiles(prev => ({ ...prev, 'scripts/generate-source-pdf.js': te...
      .catch(() => {});
  }, []);

  const activeCode = files[selectedFile] || '';
  const lines = activeCode.split('\n');

  const filteredLines = searchQuery.trim() === ''
    ? lines.map((text, idx) => ({ lineNum: idx + 1, text }))
    : lines
        .map((text, idx) => ({ lineNum: idx + 1, text }))
        .filter(item => item.text.toLowerCase().includes(searchQuery.toLowerCase()));

  const handleDownloadPdf = async () => {
    setIsDownloadingPdf(true);
    try {
      const res = await fetch('/source-code.pdf');
      if (res.ok) {
        const blob = await res.blob();
        const url = URL.createObjectURL(blob);
        const a = document.createElement('a');
        a.href = url;
        a.download = 'Dynamic-QR-Code-Source-Code.pdf';
        document.body.appendChild(a);
        a.click();
        document.body.removeChild(a);
        URL.revokeObjectURL(url);
```

```
            setDownloadSuccess(true);
            setTimeout(() => setDownloadSuccess(false), 3500);
            setIsDownloadingPdf(false);
            return;
          }
        } catch (err) {
          console.warn('Pre-generated PDF fetch error, running client generator:', err);
        }

        // Fallback using client-side jsPDF
        try {
          if (window.jspdf && window.jspdf.jsPDF) {
            const { jsPDF } = window.jspdf;
            const doc = new jsPDF({ orientation: 'portrait', unit: 'pt', format: 'a4' });
            const codeToExport = files['index.html'] || document.documentElement.outerHTML;
            const rawLines = codeToExport.split('\n');

            // Cover Page
            doc.setFillColor(79, 70, 229);
            doc.rect(0, 0, 595, 12, 'F');

            doc.setFont('helvetica', 'bold');
            doc.setFontSize(22);
            doc.setTextColor(79, 70, 229);
            doc.text('DYNAMIC QR CODE GENERATOR', 40, 70);

            doc.setFont('helvetica', 'normal');
            doc.setFontSize(12);
            doc.setTextColor(55, 65, 81);
            doc.text('Complete Standalone Web Application & Source Code Specification', 4...

            doc.setFontSize(9);
            doc.setTextColor(107, 114, 128);
            doc.text(`Generated on: ${new Date().toISOString().split('T')[0]} | Total Lin...

            doc.setDrawColor(229, 231, 235);
            doc.line(40, 130, 555, 130);

            let y = 160;
            doc.setFont('courier', 'normal');
            doc.setFontSize(7.5);
            doc.setTextColor(31, 41, 55);

            for (let i = 0; i < rawLines.length; i++) {
              if (y > 800) {
                doc.addPage();
                y = 40;
                doc.setFont('helvetica', 'normal');
                doc.setFontSize(8);
                doc.setTextColor(156, 163, 175);
                doc.text(`Dynamic QR Code Generator — Page ${doc.internal.getNumberOfPage...
                y += 20;
                doc.setFont('courier', 'normal');
                doc.setFontSize(7.5);
                doc.setTextColor(31, 41, 55);
              }
              const linePrefix = String(i + 1).padStart(4, ' ') + ' | ';
              const lineText = rawLines[i].replace(/\t/g, '  ').substring(0, 95);
              doc.text(linePrefix + lineText, 40, y);
              y += 10.5;
            }

            doc.save('Dynamic-QR-Code-Source-Code.pdf');
            setDownloadSuccess(true);
            setTimeout(() => setDownloadSuccess(false), 3500);
          } else {
            window.print();
          }
        } catch (clientErr) {
          console.error('Client PDF generation failed:', clientErr);
          window.print();
```

```
        } finally {
          setIsDownloadingPdf(false);
        }
      };

      const handleDownloadHtml = () => {
        const content = files['index.html'] || document.documentElement.outerHTML;
        const blob = new Blob([content], { type: 'text/html;charset=utf-8' });
        const url = URL.createObjectURL(blob);
        const a = document.createElement('a');
        a.href = url;
        a.download = 'dynamic-qr-code-generator.html';
        document.body.appendChild(a);
        a.click();
        document.body.removeChild(a);
        URL.revokeObjectURL(url);
      };

      const handleCopyCode = () => {
        navigator.clipboard.writeText(activeCode).then(() => {
          setCopied(true);
          setTimeout(() => setCopied(false), 2000);
        }).catch(err => {
          console.error('Failed to copy code:', err);
        });
      };

      const handlePrint = () => {
        window.print();
      };

      return React.createElement('div', { className: 'space-y-6' },
        // Top banner with back button
        React.createElement('div', { className: 'flex flex-col sm:flex-row items-start sm...
          React.createElement('div', null,
            React.createElement('div', { className: 'flex items-center gap-2 mb-1' },
              React.createElement('span', { className: 'inline-flex items-center px-2.5 p...
              React.createElement('span', { className: 'text-xs text-gray-500' }, 'Standa...
            ),
            React.createElement('h2', { className: 'text-2xl sm:text-3xl font-bold text-g...
            React.createElement('p', { className: 'text-sm text-gray-600 mt-1 max-w-2xl' },
              'Download the complete, self-contained application source code as an engine...
            )
          ),
          React.createElement('button', {
            onClick: onNavigateToGenerator,
            className: 'px-4 py-2 rounded-lg text-sm font-medium text-gray-700 bg-gray-10...
          },
            React.createElement('span', null, '? Back to Generator')
          )
        ),

        // Action Buttons Row
        React.createElement('div', { className: 'bg-indigo-50/70 border border-indigo-100...
          React.createElement('div', { className: 'flex items-center gap-3' },
            React.createElement('div', { className: 'w-10 h-10 rounded-lg bg-indigo-600 t...
              React.createElement(DocumentTextIcon, { className: 'w-6 h-6' })
            ),
            React.createElement('div', null,
              React.createElement('h3', { className: 'font-semibold text-gray-900 text-ba...
              React.createElement('p', { className: 'text-xs text-gray-600' },
                downloadSuccess
                  ? '? PDF successfully downloaded to your device!'
                  : '23 pages • Formatted Monospace • Line Numbers • File Manifest'
              )
            )
          ),
          React.createElement('div', { className: 'flex flex-wrap items-center gap-2.5' },
            React.createElement('button', {
              onClick: handleDownloadPdf,
              disabled: isDownloadingPdf,
```

```
            className: 'flex-1 sm:flex-none px-5 py-2.5 rounded-lg font-semibold text-w...
          },
            downloadSuccess
              ? React.createElement(CheckIcon, { className: 'w-4 h-4' })
              : React.createElement(ArrowDownTrayIcon, { className: 'w-4 h-4' }),
            React.createElement('span', null, isDownloadingPdf ? 'Generating PDF...' : ...
          ),
          React.createElement('button', {
            onClick: handlePrint,
            className: 'px-4 py-2.5 rounded-lg font-medium text-gray-700 bg-white borde...
          },
            React.createElement(PrinterIcon, { className: 'w-4 h-4 text-gray-500' }),
            React.createElement('span', null, 'Print / Save as PDF')
          ),
          React.createElement('button', {
            onClick: handleDownloadHtml,
            className: 'px-4 py-2.5 rounded-lg font-medium text-gray-700 bg-white borde...
          },
            React.createElement(CodeBracketIcon, { className: 'w-4 h-4 text-gray-500' }),
            React.createElement('span', null, 'Download .HTML')
          )
        )
      ),

      // Specs & Architecture Bento
      React.createElement('div', { className: 'grid grid-cols-1 sm:grid-cols-3 gap-4' },
        React.createElement('div', { className: 'p-4 bg-white rounded-xl border border-...
          React.createElement('p', { className: 'text-xs uppercase tracking-wide font-s...
          React.createElement('p', { className: 'text-xl font-bold text-gray-900 mt-1' ...
          React.createElement('p', { className: 'text-xs text-gray-500 mt-0.5' }, 'Comp...
        ),
        React.createElement('div', { className: 'p-4 bg-white rounded-xl border border-...
          React.createElement('p', { className: 'text-xs uppercase tracking-wide font-s...
          React.createElement('p', { className: 'text-xl font-bold text-gray-900 mt-1' ...
          React.createElement('p', { className: 'text-xs text-gray-500 mt-0.5' }, 'Mono...
        ),
        React.createElement('div', { className: 'p-4 bg-white rounded-xl border border-...
          React.createElement('p', { className: 'text-xs uppercase tracking-wide font-s...
          React.createElement('p', { className: 'text-xl font-bold text-gray-900 mt-1' ...
          React.createElement('p', { className: 'text-xs text-gray-500 mt-0.5' }, 'Runs...
        )
      ),

      // Code Inspector Section
      React.createElement('div', { className: 'bg-white rounded-xl border border-gray-2...
        // Top bar with file tabs and search
        React.createElement('div', { className: 'p-3 sm:p-4 bg-gray-50 border-b border-...
          // File tabs
          React.createElement('div', { className: 'flex flex-wrap gap-1.5' },
            ['index.html', 'package.json', 'metadata.json', 'scripts/generate-source-pd...
              React.createElement('button', {
                key: fileName,
                onClick: () => { setSelectedFile(fileName); setSearchQuery(''); },
                className: `px-3 py-1.5 rounded-lg text-xs font-mono font-medium transi...
              }, fileName)
            )
          ),
          // Actions: Search + Copy
          React.createElement('div', { className: 'flex items-center gap-2' },
            React.createElement('input', {
              type: 'text',
              placeholder: 'Filter code lines...',
              value: searchQuery,
              onChange: (e) => setSearchQuery(e.target.value),
              className: 'px-3 py-1.5 text-xs border border-gray-300 rounded-lg bg-whit...
            }),
            React.createElement('button', {
              onClick: handleCopyCode,
              className: 'px-3 py-1.5 rounded-lg text-xs font-medium text-gray-700 bg-w...
            },
              copied ? React.createElement(CheckIcon, { className: 'w-3.5 h-3.5 text-gr...
```

```
                React.createElement('span', null, copied ? 'Copied!' : 'Copy')
              )
            )
          ),

          // Code lines container
          React.createElement('div', {
            className: 'overflow-auto max-h-[520px] font-mono text-xs leading-relaxed bg-...
          },
            filteredLines.length > 0
              ? filteredLines.map(item =>
                  React.createElement('div', { key: item.lineNum, className: 'flex hover:...
                    React.createElement('span', { className: 'w-12 flex-shrink-0 text-gra...
                    React.createElement('pre', { className: 'flex-1 overflow-x-auto font-...
                  )
                )
              : React.createElement('div', { className: 'p-8 text-center text-gray-400' }...
          ),

          // Footer info in code box
          React.createElement('div', { className: 'px-4 py-2.5 bg-gray-100 border-t borde...
            React.createElement('span', null, `Showing ${filteredLines.length} of ${lines...
            React.createElement('span', null, 'Click "Download Source PDF" to download th...
          )
        ),

        // Print-only container for native browser print / save to PDF
        React.createElement('div', { id: 'printable-source', className: 'print-only p-4' },
          React.createElement('h1', { className: 'text-2xl font-bold mb-2' }, 'Dynamic QR...
          React.createElement('p', { className: 'text-sm text-gray-600 mb-4' }, `Generate...
          React.createElement('hr', { className: 'mb-4 border-gray-400' }),
          React.createElement('pre', { className: 'code-print-view' },
            lines.map((l, i) => `${String(i + 1).padStart(4, ' ')} | ${l}`).join('\n')
          )
        )
      );
    }


    function App() {
      const [currentView, setCurrentView] = useState('generator'); // 'generator', 'sourc...
      /** @type {[Transaction[], function(Transaction[]): void]} */
      const [transactions, setTransactions] = useState([]);
      const DEFAULT_ADMIN_PASSWORD = "063247";
      const DEFAULT_APP_SETTINGS = {
        brandName: 'Dynamic QR Code Generator',
        brandLogoUrl: '', // Can be a default URL if desired
        primaryColor: '#4f46e5', // indigo-600
        secondaryColor: '#6366f1', // indigo-500
      };

      const [adminPassword, setAdminPassword] = useState(() => {
        try {
          const storedPassword = localStorage.getItem('adminPassword');
          return storedPassword || DEFAULT_ADMIN_PASSWORD;
        } catch (error) {
          console.error("Failed to load admin password from localStorage:", error);
          return DEFAULT_ADMIN_PASSWORD;
        }
      });
      const [isAdminLoggedIn, setIsAdminLoggedIn] = useState(false);
      const [adminLoginError, setAdminLoginError] = useState('');

      /** @type {[AppSettings, function(AppSettings): void]} */
      const [appSettings, setAppSettings] = useState(() => {
        try {
          const storedSettings = localStorage.getItem('appSettings');
          return storedSettings ? JSON.parse(storedSettings) : DEFAULT_APP_SETTINGS;
        } catch (error) {
          console.error("Failed to load app settings from localStorage:", error);
          return DEFAULT_APP_SETTINGS;
```

```
        }
      });

      // Apply dynamic CSS variables on appSettings change
      useEffect(() => {
        document.documentElement.style.setProperty('--primary-color', appSettings.primary...
        document.documentElement.style.setProperty('--secondary-color', appSettings.secon...
        document.title = appSettings.brandName + " - Admin Panel"; // Update title dynami...
      }, [appSettings]);


      // Load transactions from localStorage on initial render
      useEffect(() => {
        try {
          const storedTransactions = localStorage.getItem('qrTransactions');
          if (storedTransactions) {
            setTransactions(JSON.parse(storedTransactions));
          }
        } catch (error) {
          console.error("Failed to load transactions from localStorage:", error);
        }
      }, []);

      // Save transactions to localStorage whenever they change
      useEffect(() => {
        try {
          localStorage.setItem('qrTransactions', JSON.stringify(transactions));
        } catch (error) {
          console.error("Failed to save transactions to localStorage:", error);
        }
      }, [transactions]);

      // Save admin password to localStorage whenever it changes
      useEffect(() => {
        try {
          localStorage.setItem('adminPassword', adminPassword);
        } catch (error) {
          console.error("Failed to save admin password to localStorage:", error);
        }
      }, [adminPassword]);

      // Save app settings to localStorage whenever they change
      useEffect(() => {
        try {
          localStorage.setItem('appSettings', JSON.stringify(appSettings));
        } catch (error) {
          console.error("Failed to save app settings to localStorage:", error);
        }
      }, [appSettings]);


      /**
       * @param {Omit<Transaction, 'id' | 'timestamp' | 'ipAddress' | 'type' | 'country'>...
       */
      const handleTrackTransaction = useCallback((newTransactionData) => {
        const newTransaction = {
          ...newTransactionData,
          id: crypto.randomUUID(), // Generate a unique ID
          timestamp: Date.now(),
          type: 'qrGeneration', // Set type for QR generation
          ipAddress: 'N/A (Client-side)', // Placeholder for IP address on QR gen
          country: 'N/A (Client-side)', // Placeholder for country on QR gen
        };
        setTransactions((prevTransactions) => [...prevTransactions, newTransaction]);
      }, []);

      /**
       * @param {Omit<Transaction, 'id' | 'type'> & { ipAddress: string, country: string,...
       */
      const handleTrackLicensePurchase = useCallback((licenseTransactionData) => {
        const newTransaction = {
```

```
          ...licenseTransactionData,
          id: crypto.randomUUID(), // Ensure a unique ID for the admin panel
          type: 'licensePurchase', // Set type for license purchase
          // ipAddress, country, timestamp, transactionId, status are already in licenseT...
        };
        setTransactions((prevTransactions) => [...prevTransactions, newTransaction]);
      }, []);

      /**
       * @param {string} id
       * @param {'approved' | 'denied'} newStatus
       */
      const handleUpdateLicenseStatus = useCallback((id, newStatus) => {
        setTransactions(prevTransactions =>
          prevTransactions.map(t =>
            t.id === id ? { ...t, status: newStatus } : t
          )
        );
      }, []);

      const handleClearTransactions = useCallback(() => {
        if (window.confirm('Are you sure you want to clear all transaction history? This ...
          setTransactions([]);
        }
      }, []);

      const handleAdminLogin = useCallback((passwordAttempt) => {
        if (passwordAttempt === adminPassword) {
          setIsAdminLoggedIn(true);
          setAdminLoginError('');
        } else {
          setAdminLoginError('Incorrect password.');
          setIsAdminLoggedIn(false);
        }
      }, [adminPassword]);

      const handleAdminLogout = useCallback(() => {
        setIsAdminLoggedIn(false);
        setAdminLoginError('');
        // setCurrentView('generator'); // Optionally redirect to generator on logout
      }, []);

      const handleResetAdminPassword = useCallback(() => {
        if (window.confirm('This will reset the Admin password to the default "063247" an...
          setAdminPassword(DEFAULT_ADMIN_PASSWORD);
          setIsAdminLoggedIn(false);
          setAdminLoginError('');
          alert('Admin password has been reset to "063247".');
          // setCurrentView('generator'); // Optionally redirect to generator on reset
        }
      }, []);

      /**
       * @param {AppSettings} settings
       */
      const handleSaveAppSettings = useCallback((settings) => {
        setAppSettings(settings);
      }, []);


      return React.createElement(React.Fragment, null, // Use Fragment to allow header, m...
        // Header / Navigation
        React.createElement('header', { className: 'bg-white shadow-sm p-4 border-b borde...
          React.createElement('nav', { className: 'flex flex-col sm:flex-row sm:justify-b...
            React.createElement('h1', { className: 'text-xl font-bold text-gray-800 track...
            React.createElement('div', { className: 'flex flex-wrap items-center gap-2' },
              React.createElement('button', {
                onClick: () => setCurrentView('generator'),
                className: `flex items-center gap-2 px-4 py-2 rounded-xl text-sm font-med...
              },
                React.createElement(QrCodeIcon, { className: 'w-4 h-4' }),
```

```
              React.createElement('span', null, 'Generator')
            ),
            React.createElement('button', {
              onClick: () => setCurrentView('sourceCode'),
              className: `flex items-center gap-2 px-4 py-2 rounded-xl text-sm font-med...
            },
              React.createElement(DocumentTextIcon, { className: 'w-4 h-4' }),
              React.createElement('span', null, 'Source Code PDF')
            ),
            isAdminLoggedIn && React.createElement('button', { // Only visible if admin...
              onClick: () => setCurrentView('admin'),
              className: `flex items-center gap-2 px-4 py-2 rounded-xl text-sm font-med...
            },
              React.createElement(ChartBarIcon, { className: 'w-4 h-4' }),
              React.createElement('span', null, 'Admin Panel')
            )
          )
        )
      ),

      // Main Content Area
      React.createElement('main', { className: 'flex-1 p-4 sm:p-6 lg:p-8 w-full max-w-4...
        currentView === 'generator'
          ? React.createElement(QRCodeGenerator, {
              onTrackTransaction: handleTrackTransaction,
              appSettings: appSettings,
              onNavigateToSourceCode: () => setCurrentView('sourceCode')
            })
          : currentView === 'sourceCode'
            ? React.createElement(SourceCodeExport, {
                appSettings: appSettings,
                onNavigateToGenerator: () => setCurrentView('generator')
              })
            : currentView === 'admin'
              ? (isAdminLoggedIn
                  ? React.createElement(AdminPanel, {
                      transactions: transactions,
                      onClearHistory: handleClearTransactions,
                      onUpdateLicenseStatus: handleUpdateLicenseStatus,
                      onLogout: handleAdminLogout,
                      onResetPassword: handleResetAdminPassword,
                      appSettings: appSettings,
                      onSaveAppSettings: handleSaveAppSettings // Pass save function
                    })
                  : React.createElement(AdminLogin, { onLogin: handleAdminLogin, onRese...
                )
              : React.createElement(ForSalePage, { onTrackLicensePurchase: handleTrackL...
      ),

      // Footer for Admin Panel access, Source Code export, For Sale page, and app info
      React.createElement('footer', { className: 'bg-white shadow-sm p-4 border-t borde...
        React.createElement('div', { className: 'flex flex-col sm:flex-row justify-betw...
          React.createElement('p', null, '© 2026 Dynamic QR Code Generator. https://www...
          React.createElement('div', { className: 'flex items-center gap-3' },
            React.createElement('button', {
              onClick: () => setCurrentView('sourceCode'),
              className: `flex items-center gap-1.5 px-3 py-1.5 rounded-lg font-medium ...
            },
              React.createElement(DocumentTextIcon, { className: 'w-4 h-4' }),
              React.createElement('span', null, 'Source PDF')
            ),
            React.createElement('button', {
              onClick: () => setCurrentView('admin'),
              className: `flex items-center gap-1.5 px-3 py-1.5 rounded-lg font-medium ...
            },
              React.createElement(ChartBarIcon, { className: 'w-4 h-4' }),
              React.createElement('span', null, 'Admin Panel')
            ),
            React.createElement('button', {
              onClick: () => setCurrentView('forSale'),
              className: `flex items-center gap-1.5 px-3 py-1.5 rounded-lg font-medium ...
```

```
              },
                React.createElement('span', null, 'For Sale')
              )
            )
          )
        )
      );
    }

    ReactDOM.createRoot(document.getElementById("app")).render(React.createElement(App));
  </script>
<script type="module" src="/index.tsx"></script>
</body>
</html>
```

Project Manifest — package.json

Lines 1 - 22 of 22

```
{
  "name": "dynamic-qr-code-generator",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "generate:pdf": "node scripts/generate-source-pdf.js",
    "build": "node scripts/generate-source-pdf.js && vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "jspdf": "^4.2.1",
    "pdf-lib": "^1.17.1"
  },
  "devDependencies": {
    "@types/node": "^22.14.0",
    "typescript": "~5.8.2",
    "vite": "^6.2.0"
  }
}

```

App Metadata — metadata.json

Lines 1 - 8 of 8

```
{
  "name": "Dynamic QR Code Generator",
  "description": "A modern, dynamic QR code generator that allows users to customize colo...
  "requestFramePermissions": [],
  "majorCapabilities": [
    "MAJOR_CAPABILITY_SERVER_SIDE_GEMINI_API"
  ]
}
```